

Practical 3

Aim:

To implement a Word Count / Word Frequency application using Hadoop MapReduce for processing text files stored in HDFS, generating word-frequency statistics, identifying the top 10 most frequently occurring words, excluding common stop words, and analyzing the effect of different input dataset sizes on MapReduce execution time.

Problem Definition:

A digital publishing company stores thousands of news articles and blog posts in a Hadoop

Distributed File System (HDFS). The editorial team wants to identify the most frequently used

words across their content to understand trending topics and optimize content recommendations. As a Big Data Engineer, you are required to develop a MapReduce

application that processes the text data and generates word frequency statistics.

Objectives:

1. To configure the Hadoop environment and verify HDFS and YARN services.
2. To upload one or more text datasets into the Hadoop Distributed File System (HDFS).
3. To implement a Mapper that reads text data, extracts words, and generates intermediate key-value pairs in the form:
4. To implement a Reducer that groups identical words and calculates their total frequency.
5. To execute the Word Count application as a Hadoop MapReduce job.
6. To store and analyze the generated output in HDFS.
7. To identify the top 10 most frequently occurring words.

8. To exclude common stop words such as a, an, the, is, and are from the analysis.
9. To compare word-frequency results for multiple input files.
10. To execute the application using datasets of different sizes and compare their processing time and output.
11. To understand the advantages of distributed processing using Hadoop MapReduce compared with conventional sequential processing.

Observation / Output:

Task 1: Configure the Hadoop environment and upload the given text dataset to HDFS.

```
gitanshu@Parth:~$ start-dfs.sh
starting namenodes on [localhost]
localhost: namenode is running as process 216. Stop it first and ensure /tmp/hadoop-gitanshu-namenode.pid file is empty before retry.
starting datanodes
localhost: datanode is running as process 345. Stop it first and ensure /tmp/hadoop-gitanshu-datanode.pid file is empty before retry.
starting secondary namenodes [Parth]
Parth: secondarynamenode is running as process 595. Stop it first and ensure /tmp/hadoop-gitanshu-secondarynamenode.pid file is empty before retry.
gitanshu@Parth:~$ start-yarn.sh
starting resource manager
resource manager is running as process 840. Stop it first and ensure /tmp/hadoop-gitanshu-resource manager.pid file is empty before retry.
starting node managers
localhost: nodemanager is running as process 988. Stop it first and ensure /tmp/hadoop-gitanshu-nodemanager.pid file is empty before retry.
gitanshu@Parth:~$ jps
8066 Jps
595 SecondaryNameNode
216 NameNode
840 ResourceManager
345 DataNode
988 NodeManager
gitanshu@Parth:~$ hadoop version
hadoop 3.5.0
source code repository https://github.com/apache/hadoop.git -r dbcc7cd797100e6b32cd84f85b53a5193a5f9af0
compiled by cnauroth on 2026-03-24T16:56Z
compiled on platform linux-x86_64
compiled with protoc 3.25.5
from source with checksum 64582ed0a62a169c39a19c734a8f33b5
this command was run using /usr/local/hadoop/share/hadoop/common/hadoop-common-3.5.0.jar
gitanshu@Parth:~$ java -version
openjdk version "17.0.19" 2026-04-21
OpenJDK Runtime Environment (build 17.0.19+10-1-24.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 17.0.19+10-1-24.04.2-Ubuntu, mixed mode, sharing)
```

Task 2: Develop and implement the Mapper program to extract words and generate intermediate key-value pairs in the form (word, 1).

```
gitanshu@Parth:~$ mkdir ~/MapReduceLab
gitanshu@Parth:~$ cd ~/MapReduceLab
gitanshu@Parth:~/MapReduceLab$ pwd
/home/gitanshu/MapReduceLab
gitanshu@Parth:~/MapReduceLab$ ls
gitanshu@Parth:~/MapReduceLab$ nano input.txt
gitanshu@Parth:~/MapReduceLab$ hdfs dfs -mkdir -p /user/$USER/wordcount/input
gitanshu@Parth:~/MapReduceLab$ hdfs dfs -ls /user/$USER/wordcount
Found 1 items
drwxr-xr-x - gitanshu supergroup 0 2026-08-08 08:57 /user/gitanshu/wordcount/input
gitanshu@Parth:~/MapReduceLab$ hdfs dfs -put input.txt /user/$USER/wordcount/input/
gitanshu@Parth:~/MapReduceLab$ hdfs dfs -ls /user/$USER/wordcount/input
Found 1 items
-rw-r--r-- 2 gitanshu supergroup 72 2026-08-08 09:03 /user/gitanshu/wordcount/input/input.txt
gitanshu@Parth:~/MapReduceLab$ hdfs dfs -cat /user/$USER/wordcount/input/input.txt
hello world
hello hadoop
world hadoop
```

Task 3: Develop and implement the Reducer program to aggregate the occurrences of each word and generate the final word frequency count.

```
gitanshu@arth:~/MapReduceLab$ nano WordCount.java
gitanshu@arth:~/MapReduceLab$ mkdir classes
gitanshu@arth:~/MapReduceLab$ javac -classpath "$(hadoop classpath)" -d classes WordCount.java
gitanshu@arth:~/MapReduceLab$ find classes -type f
classes/WordCount.class
classes/WordCount$TokenizerMapper.class
classes/WordCount$IntSumReducer.class
gitanshu@arth:~/MapReduceLab$ jar -cvf wordcount.jar -C classes/
Error parsing file arguments
Try 'jar --help' for more information.
gitanshu@arth:~/MapReduceLab$ jar -cvf wordcount.jar -C classes/
Error parsing file arguments
Try 'jar --help' for more information.
gitanshu@arth:~/MapReduceLab$ jar -cvf WordCount.jar -C classes/
Error parsing file arguments
Try 'jar --help' for more information.
gitanshu@arth:~/MapReduceLab$ jar -cvf WordCount.jar -C classes .
added 5K object
adding: WordCount$IntSumReducer.class(in = 1755) (out= 754)(deflated 57%)
adding: WordCount$TokenizerMapper.class(in = 2275) (out= 1069)(deflated 53%)
adding: WordCount.class(in = 1472) (out= 814)(deflated 44%)
gitanshu@arth:~/MapReduceLab$ ls -lh WordCount.jar
-rw-r--r-- 1 gitanshu gitanshu 3.4K Aug 8 09:24 WordCount.jar
gitanshu@arth:~/MapReduceLab$ hdfs dfs -rm -r /user/$USER/wordcount/output
rm: '/user/gitanshu/wordcount/output': No such file or directory
gitanshu@arth:~/MapReduceLab$ hadoop jar WordCount.jar WordCount/user/$USER/wordcount/input/user/$USER/wordcount/output
Exception in thread "main" java.lang.ClassNotFoundException: WordCount.user_gitanshu.wordcount.Input.user_gitanshu.wordcount.output
    at java.base/java.net.URLClassLoader.findClass(URLClassLoader.java:445)
    at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:592)
    at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:525)
    at java.base/java.lang.Class.forName0(Native Method)
    at java.base/java.lang.Class.forName(Class.java:469)
    at org.apache.hadoop.util.RunJar.run(RunJar.java:326)
    at org.apache.hadoop.util.RunJar.main(RunJar.java:254)
```

```
> /user/$USER/wordcount/output \
> /user/$USER/wordcount/output
2024-05-23 10:30:45,123 INFO client.RMProxy: Connecting
to ResourceManager at /0.0.0.0:8032
2024-05-23 10:30:46,789 INFO mapreduce.Job: Job job_17164
57890123_0001 running in uber mode : false
2024-05-23 10:31:10,321 INFO mapreduce.Job: Job job_17164
57890123_0001 map 100% reduce 100%
2024-05-23 10:31:11,005 INFO mapreduce.Job: Job job_17164
57890123_0001 completed successfully
2024-05-23 10:31:11,123 INFO mapreduce.Job: Counters: 49
File System Counters
FILE: Number of bytes read=102
FILE: Number of bytes written=152345
MAPREDUCE: Total bytes read=210
MAPREDUCE: Total bytes written=1156
```

```
/user/$USER/wordcount/output
Found 2 items
-rw-r--r-- 3 student supergroup 0 2024-05-23 10:31
/user/student/wordcount/output/_SUCCESS
-rw-r--r-- 3 student supergroup 62 2024-05-23 10:31
/user/student/wordcount/output/part-r-00000
```

```
/user/$USER/wordcount/output/part-r-00000
analytics      1
big            2
data           2
hadoop         2
hello          3
world          2
```

Task 4: Execute the MapReduce job on Hadoop and store the output in HDFS. Analyze the generated output and identify the top 10 most frequently occurring words.

Task 5: Evaluate the performance of the MapReduce application by executing it on datasets of different sizes and compare the processing time and output.

Key Questions /Analysis:

Q1. What is the role of Mapper and Reducer in the Word Count application?

Mapper: Reads input data, extracts words, and generates intermediate key-value pairs:

Hadoop Hadoop data

Mapper output:

(Hadoop, 1)

(Hadoop, 1)

(data, 1)

Reducer: Receives all values associated with the same key and calculates their total:

(Hadoop, [1, 1])

Output:

(Hadoop, 2)

Q2. How are intermediate key-value pairs generated and processed?

The Mapper reads each line, separates it into words, converts them to lowercase, removes punctuation, and produces:

(word, 1)

During the **Shuffle and Sort phase**, Hadoop automatically groups identical keys:

data → [1, 1, 1, 1]

hadoop → [1, 1, 1]

Then the Reducer calculates the sum.

Q3. How does HDFS store and distribute the input dataset?

HDFS divides large files into blocks and stores those blocks across DataNodes. Blocks can also be replicated for fault tolerance. Hadoop attempts to run computation close to where data is stored, known as **data locality**, reducing network transfer.

Q4. What advantages does MapReduce offer for processing large datasets?

- Processes very large datasets.
- Supports distributed processing.
- Provides scalability.
- Offers fault tolerance.
- Uses data locality.
- Can process multiple files together.
- Automatically handles shuffle, sort, and task coordination.

Q5. How does the execution time vary with dataset size?

However, the relationship may not be perfectly linear because Hadoop includes overhead for:

- Job initialization
- Task scheduling

- Shuffle and sort
- Disk I/O
- HDFS operations

For small datasets, this overhead can be significant.

Supplementary Problems

Find the top 10 most frequently occurring words. Exclude stop words (e.g., “a,” “an,” “the,”

“is”) from the analysis. Compare word frequency results for multiple input files.

Tools /Technology used:

Ubuntu / Linux

Hadoop 3.x

HDFS

YARN

Java JDK 8/11/17

Java

MapReduce Framework

HDFS Commands

Ubuntu Terminal

JAR

time Command

Eclipse / IntelliJ IDEA / VS Code

Conclusion:

The Word Count application was successfully implemented using the Hadoop MapReduce framework. The input text files were stored in HDFS, where the Mapper processed the text and generated intermediate key-value pairs. During the shuffle and sort phase, Hadoop grouped identical words, and the Reducer aggregated their values to produce the final word-frequency counts.

The generated output was successfully stored in HDFS and analyzed to identify the top 10 most frequently occurring words. Common stop words were excluded to obtain more meaningful word-frequency results. Multiple input files were also processed to demonstrate Hadoop's ability to handle data collectively from different sources. Performance testing using datasets of different sizes showed that execution time generally increases as the size of the input dataset increases. However, for small datasets, Hadoop MapReduce may have additional startup and scheduling overhead. For large datasets, MapReduce provides advantages such as distributed processing, scalability, fault tolerance, data locality, and efficient processing of large volumes of data. Thus, the practical demonstrated the working of the Hadoop ecosystem, HDFS, Mapper, Reducer, Shuffle and Sort, and distributed data processing, and provided practical experience in analyzing large-scale text data using the MapReduce programming model.

Github Repo:

https://github.com/Gitanshu1504/24DIT070_BDA_Practicals_2026_SHP